

Guía paso a paso: transiciones, transformaciones y animaciones en tu portafolio

Curso Diseño Web · Centro de Informática USS · Protech XP · Sesión 11

Tu portafolio responde al cursor: los enlaces, los botones y las tarjetas cambian con suavidad, y el Hero aparece al cargar la página. Es lo mismo que el prototipo de Figma, escrito en CSS.

Vienes de la sesión 10: necesitas las cuatro hojas de estilo (`global.css`, `header.css`, `index.css` y `contenido.css`) con el Hero, Flexbox y Grid funcionando. Si te falta algo, compáralo con la carpeta resuelta de la sesión 10 antes de empezar. Hoy no cambia el HTML: todo se escribe en CSS.

Qué vas a hacer

Hasta ahora cada regla describía un solo momento: cómo se ve la página quieta. Hoy describes dos momentos (el normal y el que aparece al pasar el cursor) y le dices al navegador cómo ir de uno a otro. Primero los estados, después las transiciones, las transformaciones y, por último, las animaciones con `@keyframes`.

Archivo	Qué cambia hoy	En Figma
<code>global.css</code>	Tres variables de movimiento y el color Hover del botón; el bloque 10 de movimiento reducido	Las medidas de la Animación inteligente
<code>header.css</code>	El subrayado animado del menú, el botón de la cabecera y los enlaces del pie	<code>nav-links</code>
<code>index.css</code>	Los botones del Hero (<code>:hover</code> y <code>:active</code>), la entrada del Hero y el cursor que parpadea	El componente <code>btn-primary</code> y su variante Hover
<code>contenido.css</code>	Enlaces, botones, campos y filas de la tabla; las tarjetas de proyecto y de contacto	<code>project-card</code> y <code>contact-card</code>

Cómo se reparte

Momento	Pasos	Qué practicas
En clase, juntos	0 al 3	Los estados en DevTools; las variables de movimiento; el menú y los botones con <code>transition</code>
En clase, tú	4 y 5	Las tarjetas con <code>transform</code> ; el Hero con <code>@keyframes</code>
Tarea	6 y 7	El resto de los estados; el movimiento reducido; validar y publicar
En Figma	Guía de Figma	El botón como componente, su variante Hover y el prototipo

Material del paquete

- **mi-portafolio-dagner/**: la carpeta resuelta de la sesión 11. El HTML es el mismo de la sesión 10.
- **Guía de Figma · Sesión 11**: el botón como componente, su variante Hover y el prototipo, clic por clic.
- **Esta guía**: cada paso indica qué archivo tocar, qué código escribir y cómo comprobarlo.

Antes de escribir: el prototipo de Figma es una transición

En la guía de Figma de hoy creas dos variantes del botón y las unes con una interacción. Cada pieza de esa interacción tiene su equivalente en CSS:

En Figma	En CSS	En tu portafolio
Variante Estado=Predeterminado	La regla normal	<code>.btn-primary { ... }</code>
Variante Estado=Hover	La misma regla con <code>:hover</code>	<code>.btn-primary:hover { ... }</code>
Desencadenante: Mientras se pasa el cursor (While hovering)	La pseudo-clase <code>:hover</code>	Al salir el cursor, vuelve sola al estado normal
Desencadenante: Mientras se presiona (While pressing)	La pseudo-clase <code>:active</code>	<code>.btn:active { ... }</code>
Acción: Cambiar a (Change to)	El navegador pasa de una regla a la otra	—
Animación: Instantáneo (Instant)	Sin <code>transition</code> : el cambio es de golpe	Así estaba tu portafolio hasta hoy
Animación: Animación inteligente (Smart animate)	<code>transition</code>	Pasa por todos los valores intermedios
Curva: Salida suave (Ease out)	<code>ease-out</code> o <code>cubic-bezier(...)</code>	<code>var(--ease-out)</code>
Duración: 200 ms	<code>0.2s</code> o <code>200ms</code>	<code>var(--duration-fast)</code>

Si no coinciden, manda Figma: el color de la variante Hover, la duración y la curva que pongas en tu prototipo son los que escribes en CSS.

Paso 0 • Los estados de un elemento

Una pseudo-clase selecciona un elemento solo mientras está en cierto estado. Ya usaste `:nth-child` y `:focus-visible` en la sesión 09. Hoy usas las de interacción:

Pseudo-clase	Cuándo se aplica	Dónde la usas
<code>:hover</code>	Mientras el cursor está encima	Menú, botones, tarjetas, filas de la tabla
<code>:active</code>	Mientras se mantiene presionado el botón del mouse o el dedo	<code>.btn:active</code>
<code>:focus</code>	Mientras el elemento tiene el foco: un campo en el que estás escribiendo	<code>input:focus</code>
<code>:focus-visible</code>	Con el foco puesto desde el teclado (Tab)	El contorno de <code>global.css</code> , desde la sesión 09

Para estudiar un estado sin perseguirlo con el mouse, fuérralo en DevTools:

1. Abre `index.html`, haz clic derecho sobre el botón «Ver Proyectos» y elige **Inspeccionar**.
2. En la pestaña **Styles**, pulsa el botón **hov**. Aparecen casillas con los estados.
3. Marca `:hover`. El botón queda en su estado Hover y en Styles aparece la regla `.btn-primary:hover`.

4. Desmárcala antes de seguir.

Un estado se escribe sin espacio: `.btn:hover` es «el botón cuando tiene el cursor encima». `.btn :hover`, con espacio, sería «cualquier elemento con el cursor encima dentro del botón».

Paso 1 • Las variables de movimiento (`assets/css/global.css`)

Al final de `:root`, debajo de los radios, añade tres variables. Así todas las transiciones del sitio usan la misma duración y la misma curva, como los estilos de Figma:

```
/* Movimiento (sesión 11): la Animación inteligente de Figma, en Salida suave y 200 ms */
--duration-fast: 0.2s;                /* cambios de color y botones */
--duration-base: 0.4s;                /* tarjetas e imágenes */
--ease-out: cubic-bezier(0.22, 1, 0.36, 1); /* arranca rápido y frena suave */
}
```

Y debajo de `--color-border`, el color de la variante Hover del botón:

```
--color-primary-hover: #05A3B0; /* Sesión 11: variante Hover del botón en Figma */
```

Curva (timing function)	Cómo se mueve	Cuándo usarla
linear	A la misma velocidad de principio a fin	Giros continuos y barras de progreso
ease	Arranca suave, acelera y frena (el valor por defecto)	Cambios de color
ease-in	Arranca lento y termina rápido	Elementos que salen de la pantalla
ease-out	Arranca rápido y frena al llegar	Respuestas al cursor: se sienten inmediatas
ease-in-out	Lento al principio y al final	Movimientos largos de un lugar a otro
cubic-bezier(a, b, c, d)	Una curva a medida, con cuatro números	<code>--ease-out</code> : una salida suave más marcada
steps(n)	Salta en n pasos, sin valores intermedios	El cursor que parpadea (Paso 5)

¿Cuánto debe durar? entre 150 y 400 milisegundos. Menos de 100 no se percibe y más de 500 hace que la página parezca lenta. Los botones, rápidos (0.2s); las piezas grandes, como una tarjeta, un poco más (0.4s).

Paso 2 • El menú con `transition` (`assets/css/header.css`)

`transition` se escribe en la regla **normal**, no en la de `:hover`. Así el cambio es suave al entrar y también al salir el cursor.

Propiedad	Qué indica	Ejemplo
transition-property	Qué propiedad se anima	color

Propiedad	Qué indica	Ejemplo
transition-duration	Cuánto dura	0.2s
transition-timing-function	Con qué curva	ease-out
transition-delay	Cuánto espera antes de empezar (opcional)	0.1s
transition (abreviada)	Las cuatro en una línea; varias propiedades se separan con comas	color 0.2s ease, transform 0.4s ease-out

Reemplaza la regla de los enlaces del menú (bloque 3) y añade su `:hover`:

```
nav[aria-label="Navegacion principal"] a {
  font-size: 14px;
  font-weight: 500;
  color: var(--color-text-muted);
  padding-bottom: 4px;
  /* Subrayado animado: un degradado de un solo color como imagen de fondo (sesión 09).
     Empieza con 0 % de ancho y crece hasta el 100 % al pasar el cursor */
  background-image: linear-gradient(var(--color-primary), var(--color-primary));
  background-size: 0% 2px;
  background-position: left bottom;
  background-repeat: no-repeat;
  transition: color var(--duration-fast) ease,
             background-size var(--duration-base) var(--ease-out);
}

nav[aria-label="Navegacion principal"] a:hover {
  color: var(--color-text-primary);
  background-size: 100% 2px;
}
```

El subrayado es una imagen de fondo: un degradado de un solo color (sesión 09). En reposo mide 0% de ancho y 2 px de alto; con el cursor encima, 100%. Como `background-size` es una medida, se puede animar.

No todo se puede animar: se animan las propiedades que tienen valores intermedios: colores, medidas, opacity, transform, box-shadow. display o font-family cambian de golpe aunque tengan transition.

Haz lo mismo con el botón de la cabecera (bloque 4) y los enlaces del pie (bloque 5):

```
header > a[href="contactame.html"] {
  display: inline-flex;
  align-items: center;
  padding: 14px 24px;
  border: 1px solid var(--color-text-secondary);
  border-radius: var(--radius-sm);
  font-size: 15px;
  font-weight: 500;
  color: var(--color-text-primary);
  transition: background-color var(--duration-fast) ease,
             color var(--duration-fast) ease;
}

header > a[href="contactame.html"]:hover {
  background-color: var(--color-text-primary);
  color: #FFFFFF;
}
```

```

footer a {
  transition: color var(--duration-fast) ease;
}

footer a:hover {
  color: var(--color-primary);
}

```

Comprueba: al pasar el cursor por el menú, el texto se oscurece y una línea azul crece de izquierda a derecha. Al retirarlo, la línea se recoge.

Paso 3 • Los botones del Hero: transition y transform (assets/css/index.css)

t transform mueve, escala o gira un elemento **después** de colocarlo en la página. Sus vecinos no se enteran: no se empujan ni se recolocan. Por eso es la mejor propiedad para animar.

Función	Qué hace	Ejemplo
translateX(n) • translateY(n) • translate(x, y)	Desplaza en horizontal, en vertical o en los dos ejes	translateY(-2px): sube 2 px
scale(n)	Agranda o achica; 1 es el tamaño normal	scale(1.05): un 5 % más grande
rotate(n)	Gira en grados (deg); negativo, al revés del reloj	rotate(-10deg)
skew(n)	Inclina	skewX(-8deg)
Varias funciones	Se escriben separadas por espacios y se aplican en orden	translateY(0) scale(0.98)
transform-origin	El punto desde el que se escala o se gira; por defecto, el centro	transform-origin: left;

Reemplaza el bloque 3 de index.css:

```

/* 3. Botones de llamada a la acción • Figma: hero-actions, Espacio 16 */
.hero-actions {
  display: flex;
  gap: 16px;
  align-items: center;
}

.btn {
  display: inline-flex; /* el enlace deja de ser inline: acepta padding vertical */
  align-items: center;
  justify-content: center;
  padding: 14px 24px; /* Espaciado 14 y 24 */
  border-radius: var(--radius-sm); /* Radio de esquina 8 */
  font-size: 15px;
  font-weight: 600;
  transition: background-color var(--duration-fast) ease,
    border-color var(--duration-fast) ease,
    box-shadow var(--duration-fast) ease,
    transform var(--duration-fast) var(--ease-out);
}

.btn-primary {
  background-color: var(--color-primary);
}

```

```

    color: #FFFFFF;
    border: 1px solid var(--color-primary);
}

.btn-primary:hover {
    background-color: var(--color-primary-hover); /* Figma: Relleno de la variante Hover */
    border-color: var(--color-primary-hover);
    box-shadow: 0 4px 14px rgba(6, 182, 196, 0.35);
    transform: translateY(-2px); /* sube 2 px: no mueve a los demás elementos */
}

.btn-secondary {
    background-color: transparent;
    color: var(--color-text-primary);
    border: 1px solid var(--color-text-secondary);
}

.btn-secondary:hover {
    background-color: rgba(11, 15, 25, 0.05);
}

/* Al pulsar, el botón se encoge un poco: confirma el clic.
   Va después de los :hover para ganarles por orden (misma especificidad) */
.btn:active {
    transform: translateY(0) scale(0.98);
}

```

El orden importa: `.btn:active` y `.btn-primary:hover` tienen la misma especificidad (0-2-0). Si las dos se cumplen (el cursor está encima y además presionas), gana la que está escrita después. Por eso `:active` va al final.

¿Por qué no `margin-top: -2px`? porque `margin` sí empuja a los vecinos: el párrafo de arriba y el otro botón se moverían con él. `transform` mueve solo al botón.

Comprueba: con el cursor encima, «Ver Proyectos» se oscurece, sube 2 px y proyecta una sombra azul. Al presionarlo, se encoge un poco. «Contáctame» se rellena con un gris muy claro.

Paso 4 • Las tarjetas con transform (assets/css/contenido.css)

4.1 • Las tarjetas de proyecto

En el bloque 13.3, añade `transition` a `.project-card` y a `.project-image`, y escribe sus `:hover`:

```

/* Figma: project-card, Espacio 0, Espaciado 0, Recortar contenido */
.project-card {
    padding: 0; /* anula el 1.5rem de las tarjetas: la imagen va de borde a borde */
    /*
    margin-bottom: 0; /* en la rejilla, la separación la pone gap */
    overflow: hidden; /* Recortar contenido: la imagen respeta las esquinas */
    transition: transform var(--duration-base) var(--ease-out),
                box-shadow var(--duration-base) var(--ease-out);
    }

/* Sesión 11: al pasar el cursor, la tarjeta sube y proyecta sombra */
.project-card:hover {
    transform: translateY(-6px);
    box-shadow: 0 16px 32px rgba(11, 15, 25, 0.08);
}

```

```

.project-image {
  display: block;          /* una imagen es inline: deja un hueco debajo si no se cambia */
  width: 100%;
  height: 220px;          /* Figma: rectángulo de 220 de alto */
  object-fit: cover;      /* llena la caja sin deformarse, como el modo Llenar de Figma */
  border-radius: 0;        /* la clase gana a la regla de img: las esquinas las recorta la
tarjeta */
  transition: transform var(--duration-base) var(--ease-out);
}

/* La imagen se acerca un 5 %; overflow: hidden de la tarjeta recorta lo que sobra */
.project-card:hover .project-image {
  transform: scale(1.05);
}

```

El selector `.project-card:hover .project-image` se lee «la imagen que está dentro de una tarjeta que tiene el cursor encima». El cursor puede estar sobre el texto de la tarjeta, no sobre la imagen, y aun así la imagen se acerca. El `overflow: hidden` de la sesión 10 recorta lo que sobra: la imagen crece dentro de su marco.

4.2 • Las tarjetas de contacto

En el bloque 13.4, añade `transition` a `.contact-card` y a `.contact-icon` y sus reglas `:hover`:

```

/* Figma: contact-card, Flujo →, Espaciado 20, Espacio 16, Alineación centro */
.contact-card {
  display: flex;
  align-items: center;
  gap: 16px;
  padding: 20px;
  margin-bottom: 0;        /* la clase gana a «main li» */
  background-color: var(--color-surface);
  border: 1px solid var(--color-border);
  border-radius: var(--radius-sm);
  transition: border-color var(--duration-fast) ease,
              transform var(--duration-fast) var(--ease-out);
}

.contact-card:hover {
  border-color: var(--color-primary);
  transform: translateX(4px); /* se desplaza 4 px a la derecha */
}

```

```

/* El círculo de 44 × 44: flex también sirve para centrar en los dos ejes */
.contact-icon {
  flex-shrink: 0;          /* nunca se aplasta aunque el texto sea largo */
  width: 44px;
  height: 44px;
  border-radius: 50%;
  display: flex;
  align-items: center;
  justify-content: center;
  background-color: var(--color-primary);
  color: #FFFFFF;
  font-weight: 700;
  font-size: 0.875rem;
  transition: transform var(--duration-base) var(--ease-out);
}

.contact-card:hover .contact-icon {
  transform: rotate(-10deg) scale(1.08); /* gira desde su centro: transform-origin por

```

```
defecto */  
}
```

Comprueba: en Proyectos, la tarjeta sube 6 px con una sombra y su imagen se acerca. En Contacto, la fila se desplaza 4 px a la derecha, su borde se vuelve azul y el círculo gira un poco.

Paso 5 • Animaciones con @keyframes (assets/css/index.css)

Una transición necesita un cambio de estado: el cursor entra o sale. Una **animación** arranca sola, al cargar la página, y puede repetirse. Se escribe en dos partes: los fotogramas clave, con un nombre, y la propiedad animation que los usa.

5.1 • Los fotogramas clave

Al final de index.css, antes de las media queries, crea el bloque 5:

```
/* 5. Animaciones con @keyframes (sesión 11) */  
@keyframes aparecer {  
  from {  
    opacity: 0;  
    transform: translateY(24px); /* empieza 24 px más abajo */  
  }  
  to {  
    opacity: 1;  
    transform: translateY(0);  
  }  
}  
  
@keyframes parpadeo {  
  50% { opacity: 0; } /* a la mitad del ciclo se apaga */  
}
```

from es el 0 % de la animación y to, el 100 %. Entre los dos puedes poner los porcentajes que quieras, como el 50% de parpadeo. Las propiedades que no se escriben en un fotograma conservan su valor normal.

5.2 • La entrada del Hero

En el bloque 2, añade la animación a .hero-left, y en el bloque 4, a .hero-right con 0.2 s de retraso:

```
.hero-left {  
  flex: 1; /* Llenar el contenedor */  
  max-width: 696px; /* Ancho fijo 696, pero puede encogerse */  
  display: flex;  
  flex-direction: column; /* Flujo ↓ */  
  gap: 32px;  
  animation: aparecer 0.8s var(--ease-out) both; /* entrada al cargar la página */  
}
```

```
.hero-right {  
  flex: 1;  
  max-width: 520px; /* Ancho fijo 520 */  
  display: flex;  
  justify-content: center;  
  animation: aparecer 0.8s var(--ease-out) 0.2s both; /* entra 0.2 s después */  
}
```

Propiedad	Qué indica	En aparecer
animation-name	Qué @keyframes usar	aparecer
animation-duration	Cuánto dura una vuelta	0.8s
animation-timing-function	Con qué curva	var(--ease-out)
animation-delay	Cuánto espera antes de empezar	0.2s en .hero-right
animation-iteration-count	Cuántas veces se repite; infinite, sin fin	1 (por defecto)
animation-direction	normal, reverse o alternate (ida y vuelta)	normal
animation-fill-mode	Qué valores muestra antes de empezar y al terminar	both

¿Para qué sirve both? sin él, la columna derecha se vería quieta durante los 0.2 s de espera y después saltaría a opacity: 0 para empezar a aparecer. Con both, muestra el fotograma from mientras espera y se queda en el to al terminar.

5.3 • El cursor que parpadea

En el bloque 4, después de .code-body, añade un pseudo-elemento al final del código:

```
/* El cursor que parpadea al final del código */
.code-body code::after {
  content: "";
  display: inline-block;
  width: 8px;
  height: 1.1em;
  margin-left: 4px;
  vertical-align: text-bottom;
  background-color: var(--color-primary);
  animation: parpadeo 1s steps(1) infinite;
}
```

::after crea una caja vacía al final del <code>: no hace falta tocar el HTML. steps(1) hace que cambie de golpe, sin pasar por la mitad de la opacidad: se enciende y se apaga como el cursor de un editor.

Comprueba: al recargar la portada, la columna de texto sube y aparece, y la tarjeta de código la sigue un instante después. Al final del código parpadea un cursor azul.

Paso 6 • El resto de los estados (assets/css/contenido.css)

Repasa las páginas interiores y añade transition donde ya había un cambio de estado de la sesión 09:

```

/* 4. Enlaces del contenido: recuperan el subrayado que quitó global.css */
main a {
  color: var(--color-text-primary);
  text-decoration: underline;
  text-decoration-color: var(--color-primary);
  text-underline-offset: 3px;
  transition: color var(--duration-fast) ease;
}

main a:hover {
  color: var(--color-primary);
}

```

```

/* 8. Botones: no heredan la letra de la página, hay que pedirselo */
button {
  font: inherit;
  font-weight: 600;
  padding: 0.875rem 1.5rem; /* 14px y 24px, como los botones de Figma */
  border: 1px solid var(--color-text-secondary);
  border-radius: var(--radius-sm);
  background-color: var(--color-surface);
  color: var(--color-text-primary);
  cursor: pointer;
  transition: background-color var(--duration-fast) ease,
              color var(--duration-fast) ease,
              box-shadow var(--duration-fast) ease;
}

button:hover {
  background-color: var(--color-text-primary);
  color: #FFFFFF;
}

/* Selector de atributo: solo el botón que envía el formulario */
button[type="submit"] {
  background-color: var(--color-primary);
  border-color: var(--color-primary);
  color: #FFFFFF;
}

button[type="submit"]:hover {
  background-color: var(--color-primary-hover);
  border-color: var(--color-primary-hover);
  box-shadow: 0 4px 14px rgba(6, 182, 196, 0.35); /* el azul de Figma al 35 % */
}

```

```

/* en la regla de input, select y textarea (bloque 9), al final */
transition: border-color var(--duration-fast) ease;

input:focus,
select:focus,
textarea:focus {
  border-color: var(--color-primary);
}

```

```

tbody tr {
  transition: background-color var(--duration-fast) ease;
}

tbody tr:hover {
  background-color: rgba(6, 182, 196, 0.08); /* el azul de Figma al 8 % */
}

```

Comprueba: los enlaces, los botones del formulario, los campos al hacer clic en ellos y las filas de la tabla cambian con suavidad.

Paso 7 • Movimiento reducido, validación y publicación

7.1 • Respeta a quien pidió menos movimiento

Algunas personas se marean con las animaciones y lo indican en su sistema operativo (en Windows: **Configuración > Accesibilidad > Efectos visuales > Efectos de animación**). La media query `prefers-reduced-motion` lo detecta. Añade el bloque 10 al final de `global.css`:

```
/* 10. Movimiento reducido: si la persona pidió menos animaciones en su sistema,
   la página se muestra quieta. Es la excepción en la que usamos !important:
   esta regla tiene que ganar a todas las transiciones y animaciones del sitio */
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
  }
}
```

Las duraciones pasan a 0.01 ms: todo cambia de golpe y las animaciones terminan en cuanto empiezan, así que el Hero aparece directamente en su sitio. No se escribe 0 porque algunos navegadores entonces no avisan del final de la animación.

!important es la excepción: gana a cualquier otra regla, sin importar la especificidad ni el orden. No lo uses para arreglar una regla que no se aplica; aquí sí tiene sentido, porque este bloque debe ganar a todas las transiciones del sitio.

Para probarlo sin cambiar tu sistema: en DevTools pulsa `Ctrl + Shift + P`, escribe **rendering** y elige **Show Rendering**. En la pestaña que se abre, en **Emulate CSS media feature prefers-reduced-motion**, elige `reduce` y recarga la página.

7.2 • Valida y publica

1. Valida tus cuatro hojas de estilo en jigsaw.w3.org/css-validator: cero errores.
2. Valida las cuatro páginas en validator.w3.org: como el HTML no cambió, deben seguir sin errores.
3. Recorre las cuatro páginas solo con el teclado (Tab): cada enlace y botón muestra su contorno.
4. Publica en Netlify y prueba los efectos en tu celular: allí no hay cursor, así que `:hover` se activa al tocar.

Lista de comprobación

- ☐ `:root` tiene `--duration-fast`, `--duration-base`, `--ease-out` y `--color-primary-hover`.
- ☐ Cada `transition` está en la regla normal, no en la de `:hover`.
- ☐ Ninguna transición usa `all`: cada una nombra las propiedades que cambian.
- ☐ Los movimientos usan `transform`, no `margin` ni `top`.
- ☐ El botón principal se oscurece con el mismo color que la variante `Hover` de tu Figma.
- ☐ `.btn:active` está escrito después de los `:hover`.
- ☐ El Hero aparece al cargar la portada y el cursor del código parpadea.

- ☐ Con prefers-reduced-motion: reduce, la página se ve quieta.
- ☐ El CSS pasa el validador del W3C sin errores.

Errores frecuentes en esta sesión

Síntoma	Causa y solución
El cambio es suave al entrar y brusco al salir	La transition está en la regla :hover. Muévela a la regla normal
No hay transición aunque está escrita	El nombre de la propiedad no coincide (background en una y background-color en la otra) o la propiedad no se puede animar, como display
Al pasar el cursor, los elementos de al lado se mueven	Se usó margin o top para mover. Cámbialo por transform: translateY(...)
:active no hace nada	Está escrito antes que el :hover con la misma especificidad. Pásalo debajo
La animación no arranca	El nombre de animation no es igual al de @keyframes (mayúsculas, tildes) o falta la duración: sin ella dura 0 s
El Hero parpadea antes de aparecer	Falta both en la columna con retraso: espera visible y después salta a opacity: 0
La imagen de la tarjeta se sale al acercarse	Falta overflow: hidden; en .project-card
El cursor del código no se ve	Al ::after le falta content: "" o display: inline-block: sin ellos no hay caja que pintar